

Python 初級教學

1. 輸出 Hello World 訊息
2. 資料型別
 - 整數 (integer)
 - 浮點數 (float)
 - 布林值 (Boolean)
 - 字串 (string)
3. 變數
4. 列表 (Lists)
5. 字典 (Dictionaries)
6. 條件判斷 (If Statement)
7. 迴圈 (For-Loop)
8. 函式 (Functions)
9. 模組 (Modules)

1. 輸出 Hello World 訊息

```
In [1]: # 輸出我的第一條訊息  
print('Hello World')
```

Hello World

2. 資料型別 (Data Types)

```
In [2]: print(type(1))
```

<class 'int'>

```
In [3]: print(type(2.4))
```

<class 'float'>

```
In [4]: print(1 + 2)
```

3

```
In [5]: print("4-1")
```

4-1

```
In [6]: print(type(True))
```

<class 'bool'>

```
In [7]: print(type(False))
```

<class 'bool'>

```
In [8]: print(type('Hello World'))
```

```
<class 'str'>
```

3. 變數 (Variables)

```
In [9]: message_1 = "I'm learning Python"
print("message_1=", message_1)
```

```
message_1= I'm learning Python
```

```
In [10]: message_2 = "and it's fun"
print(message_2)
```

```
and it's fun
```

```
In [11]: message = message_1 + " " + message_2
print(message)
```

```
I'm learning Python and it's fun
```

```
In [12]: # 另一種格式化輸出的方式
print(f'message_1: {message_1}\nmessage_2: {message_2}')
```

```
message_1: I'm learning Python
```

```
message_2: and it's fun
```

4. 列表 (Lists)

- 新增元素到列表
- 從列表中移除元素
- 訪問列表中的元素
- 列表切片
- 在列表中插入元素
- 合併列表
- 巢狀列表
- 列表排序
- 複製列表

```
In [13]: countries = ['United States', 'India', 'Taiwan', 'Brazil']
print(countries)
```

```
# 在列表末尾添加一個元素
countries.append('Japan')
print(countries)
```

```
# 從列表中移除一個元素
countries.remove('Brazil')
print(countries)
```

```
['United States', 'India', 'Taiwan', 'Brazil']
```

```
['United States', 'India', 'Taiwan', 'Brazil', 'Japan']
```

```
['United States', 'India', 'Taiwan', 'Japan']
```

```
In [14]: # 更多移除列表元素的方法

countries = ['United States', 'India', 'Taiwan', 'Brazil']
print(countries)

# 移除並返回最後一個元素
removed_country = countries.pop()
print(removed_country)

# 移除倒數第一個元素
countries.pop(-1)
print(countries)

# 刪除指定索引的元素
del countries[0]
print(countries)

['United States', 'India', 'Taiwan', 'Brazil']
Brazil
['United States', 'India']
['India']
```

```
In [15]: # 存取列表中的元素 (索引從0開始)
countries = ['United States', 'India', 'Taiwan', 'Brazil']
print(countries[0])
```

United States

```
In [16]: # 使用負數索引從列表末尾開始計數
countries = ['United States', 'India', 'Taiwan', 'Brazil']
print(countries[-1])
```

Brazil

```
In [17]: countries = ['United States', 'India', 'Taiwan', 'Brazil']
print(countries[-4])
```

United States

```
In [18]: # 列表切片 [起始索引:結束索引]
countries = ['United States', 'India', 'Taiwan', 'Brazil']
print(countries[0:2])
```

['United States', 'India']

```
In [19]: countries = ['United States', 'India', 'Taiwan', 'Brazil']
print(countries[1:])
```

['India', 'Taiwan', 'Brazil']

```
In [20]: countries = ['United States', 'India', 'Taiwan', 'Brazil']
print(countries[:2])
```

['United States', 'India']

```
In [21]: # 在列表中插入元素
countries = ['United States', 'India', 'Taiwan', 'Brazil']
countries.insert(0, 'Spain')
```

```
print(countries)
```

```
['Spain', 'United States', 'India', 'Taiwan', 'Brazil']
```

```
In [22]: # 列表合併
countries = ['United States', 'India', 'Taiwan', 'Brazil']
countries_2 = ["Finland", "Sweden", "Denmark", "Norway", "Iceland"]
print(countries + countries_2)
```

```
['United States', 'India', 'Taiwan', 'Brazil', 'Finland', 'Sweden',
'Denmark', 'Norway', 'Iceland']
```

```
In [23]: # 巢狀列表 (列表中的列表)
countries = ['United States', 'India', 'Taiwan', 'Brazil']
countries_2 = ['China', 'Japan', 'South Korea', 'Vietnam']
nested_list = [countries, countries_2]
print(nested_list)
```

```
[['United States', 'India', 'Taiwan', 'Brazil'], ['China', 'Japan',
'South Korea', 'Vietnam']]
```

```
In [24]: # 對列表進行排序
numbers = [4, 3, 10, 9, 8, 2]
print(numbers)
numbers.sort()
print(numbers)
```

```
[4, 3, 10, 9, 8, 2]
[2, 3, 4, 8, 9, 10]
```

```
In [25]: # 降序排序列表
numbers = [4, 3, 10, 9, 8, 2]
numbers.sort(reverse=True)
print(numbers)
```

```
[10, 9, 8, 4, 3, 2]
```

```
In [26]: # 更新列表中的元素
numbers = [4, 3, 10, 9, 8, 2]
print(numbers)

numbers[0] = 1000
print(numbers)
```

```
[4, 3, 10, 9, 8, 2]
[1000, 3, 10, 9, 8, 2]
```

```
In [27]: # 複製列表 (使用切片)
countries = ['United States', 'India', 'Taiwan', 'Brazil']
new_list = countries[:]
print(new_list)
```

```
['United States', 'India', 'Taiwan', 'Brazil']
```

```
In [28]: # 另一種複製列表的方法 (使用copy()方法)
new_list2 = countries.copy()
print(new_list2)
```

```
['United States', 'India', 'Taiwan', 'Brazil']
```

5. 字典 (Dictionaries)

1. 複製字典
2. 從字典中移除元素

```
In [29]: my_dict = {'key1': 'value1', 'key2': 'value2'}  
print(my_dict)
```

```
{'key1': 'value1', 'key2': 'value2'}
```

```
In [30]: my_data = {'name': 'Michael', 'age': 63}  
print(my_data)
```

```
{'name': 'Michael', 'age': 63}
```

```
In [31]: print(my_data.keys())
```

```
dict_keys(['name', 'age'])
```

```
In [32]: print(my_data.values())
```

```
dict_values(['Michael', 63])
```

```
In [33]: my_data.items()
```

```
Out[33]: dict_items([('name', 'Michael'), ('age', 63)])
```

```
In [34]: my_data['height'] = 165  
print(my_data)
```

```
{'name': 'Michael', 'age': 63, 'height': 165}
```

```
In [35]: my_data.update({'height': 180})  
print(my_data)
```

```
{'name': 'Michael', 'age': 63, 'height': 180}
```

5.1 Copy a dictionary

```
In [36]: my_data = {'name': 'Michael', 'age': 63, 'height': 165}  
print(my_data)
```

```
new_dict = my_data.copy() # 複製資料  
print(f'new_dict: {new_dict}') # 不會影響到另一個資料副本
```

```
new_dict2 = my_data # 複製參考  
print(f'new_dict2: {new_dict2}') # 指向同一份資料的參考
```

```
{'name': 'Michael', 'age': 63, 'height': 165}  
new_dict: {'name': 'Michael', 'age': 63, 'height': 165}  
new_dict2: {'name': 'Michael', 'age': 63, 'height': 165}
```

```
In [37]: my_data = {'name': 'Michael', 'age': 63, 'height': 165}  
print(f'my_data: {my_data}')
```

```
new_dict = my_data.copy() # 複製資料
```

```
new_dict2 = my_data # 複製參考

my_data.update({'height':180}) # 新增身高資料
print(f'my_data: {my_data}')

print(f'new_dict: {new_dict}') # 不會影響到另一個資料副本
print(f'new_dict2: {new_dict2}') # 指向同一份資料的參考
```

```
my_data: {'name': 'Michael', 'age': 63, 'height': 165}
my_data: {'name': 'Michael', 'age': 63, 'height': 180}
new_dict: {'name': 'Michael', 'age': 63, 'height': 165}
new_dict2: {'name': 'Michael', 'age': 63, 'height': 180}
```

5.2 從字典中移除元素

```
In [38]: my_data = {'name': 'Michael', 'age': 63, 'height': 180}
print(my_data)
my_data.pop('name') # 使用鍵來移除元素
my_data
```

```
{'name': 'Michael', 'age': 63, 'height': 180}
```

```
Out[38]: {'age': 63, 'height': 180}
```

```
In [39]: del my_data['age'] # 使用del刪除指定鍵的元素
my_data
```

```
Out[39]: {'height': 180}
```

```
In [40]: my_data.clear() # 清除字典中所有元素
my_data
```

```
Out[40]: {}
```

6. 條件判斷 (If Statement)

語法 (Syntax):

```
if (條件):
    # 程式區塊
else:
    # 程式區塊
```

```
In [41]: age = 10

if age >= 18:
    print("你是成年人!")
elif age >= 13:
    print("你是青少年。")
else:
    print("你是小孩。")
```

你是小孩。

7. 迴圈 (For-loop)

語法 (Syntax):

```
for i in range(10):  
    print(i)
```

```
In [42]: countries = ['United States', 'India', 'Taiwan', 'Brazil']  
  
countries
```

```
Out[42]: ['United States', 'India', 'Taiwan', 'Brazil']
```

```
In [43]: for country in countries:  
         print(country)
```

```
United States  
India  
Taiwan  
Brazil
```

```
In [44]: for i, country in enumerate(countries):  
         print(i)  
         print(country)
```

```
0  
United States  
1  
India  
2  
Taiwan  
3  
Brazil
```

```
In [45]: my_data = {'name': 'Michael', 'age': 63, 'height': 180}  
  
my_data
```

```
Out[45]: {'name': 'Michael', 'age': 63, 'height': 180}
```

```
In [46]: for key, value in my_data.items():  
         print(key)  
         print(value)
```

```
name  
Michael  
age  
63  
height  
180
```

8. 函式 (Functions)

1. 內建函式 (Built-in functions)

2. 自訂函式 (User-defined Functions)

語法 (Syntax):

```
def 函式名稱(參數):  
    <程式碼>  
    return 回傳值
```

```
In [47]: def add(a, b):  
         return a+b
```

```
In [48]: add(1,3)
```

```
Out[48]: 4
```

8.1 內建函式 (Built-in functions)

Python 提供了許多內建函式，用於執行各種操作。例如：len(), max(), min(), print(), type(), etc.

```
In [49]: countries = ['United States', 'India', 'Taiwan', 'Brazil']  
         countries
```

```
Out[49]: ['United States', 'India', 'Taiwan', 'Brazil']
```

```
In [50]: len(countries)
```

```
Out[50]: 4
```

```
In [51]: max([10,3, 6, 33, 100])
```

```
Out[51]: 100
```

```
In [52]: min([10,3, 6, 33, 100])
```

```
Out[52]: 3
```

```
In [53]: type(countries)
```

```
Out[53]: list
```

```
In [54]: print('Hello World'.upper())  
         print('Hello World'.lower())  
         print('Hello World'.title())  
         print('Hello World'.count('l'))  
         print('Hello World'.replace('o', '0'))
```

```
HELLO WORLD
hello world
Hello World
3
Hello W0rld
```

8.2 自訂函式 (User-defined Functions)

下面範例程式可以幫助您快速了解系統當前的記憶體使用狀況，包括總記憶體、可用記憶體、已用記憶體以及記憶體使用率。

說明：

1. psutil 模組

- 用於獲取系統資源使用情況
- `psutil.virtual_memory()` 返回系統記憶體使用情況的命名元組

2. convert_bytes 函數

- 功能：將位元組(bytes)轉換為GB
- 參數：size - 要轉換的位元組數
- 返回值：轉換後的GB數值（浮點數）

3. 記憶體變數說明：

- `memory.total`：系統總記憶體（位元組）
- `memory.available`：可用記憶體（位元組）
- `memory.used`：已使用記憶體（位元組）
- `memory.percent`：記憶體使用百分比

4. 輸出格式：

- 記憶體大小顯示到小數點後三位
- 使用f-string進行格式化輸出

```
In [55]: import psutil

# 獲取系統記憶體資訊
memory = psutil.virtual_memory()

def convert_bytes(size):
    """將位元組(bytes)轉換為GB

    Args:
        size (int): 位元組大小

    Returns:
        float: 轉換後的GB數值
    """
    # 將位元組轉換為GB (1024^3 bytes = 1 GB)
    gb = size / (1024 ** 3)
```

```

return gb

# 轉換記憶體單位
total_gb = convert_bytes(memory.total)      # 系統總記憶體
available_gb = convert_bytes(memory.available) # 可用記憶體
used_gb = convert_bytes(memory.used)         # 已使用記憶體

# 輸出記憶體使用情況
print(f"總記憶體: {total_gb:.3f} GB")
print(f"可用記憶體: {available_gb:.3f} GB")
print(f"已用記憶體: {used_gb:.3f} GB")
print(f"記憶體使用率: {memory.percent}%")

```

```

總記憶體: 16.000 GB
可用記憶體: 3.838 GB
已用記憶體: 6.129 GB
記憶體使用率: 76.0%

```

8.3 Positional and Keyword Arguments

Python functions can accept arguments in two ways: positional arguments and keyword arguments. Positional arguments are arguments that are passed to a function in the order in which they are defined. Keyword arguments are arguments that are passed to a function by name.

注意事項:

- 位置參數必須按照函數定義的順序傳遞
- 必要的位置參數必須在呼叫時提供
- 有預設值的參數應該放在沒有預設值的參數之後
- 位置參數在函數調用時不能重複指定

範例展示:

```

def greet(name, greeting):
    """這是一個簡單的問候函數

    Args:
        name (str): 要問候的人名
        greeting (str): 問候語

    Returns:
        str: 完整的問候訊息
    """
    return f"{greeting}, {name}!"

# 使用位置參數呼叫函數
message = greet("小明", "你好") # 參數按照定義的順序傳遞
print(message) # 輸出: 你好, 小明!

```

```

In [56]: # 多個位置參數
def calculate_rectangle_area(length, width):
    """計算長方形的面積

```

```

    Args:
        length (float): 長度
        width (float): 寬度

    Returns:
        float: 長方形面積
    """
    return length * width

# 呼叫函數時，參數必須按照定義的順序
area = calculate_rectangle_area(5.5, 3.2)
print(f"長方形面積: {area} 平方單位") # 輸出: 長方形面積: 17.6 平方單位

# 位置參數的強制性
# 位置參數是必須的，如果呼叫時沒有提供足夠的參數，會引發 TypeError :
try:
    area = calculate_rectangle_area(5.5) # 引發 TypeError
except TypeError as e:
    print(e)

# 位置參數與預設值結合
# 位置參數也可以有預設值，但有預設值的參數必須放在沒有預設值的參數之後：
def power(base, exponent=2):
    """計算冪次方

    Args:
        base (int or float): 底數
        exponent (int or float, optional): 指數，預設為2

    Returns:
        int or float: 計算結果
    """
    return base ** exponent

# 使用預設指數
print(f'power(3)\t={power(3)}') # 輸出: 9 (3的2次方)

# 指定指數
print(f'power(3, 3)\t={power(3, 3)}') # 輸出: 27 (3的3次方)

# positional arguments becomes keyword arguments
print(f'power(base=3, exponent=3)\t={power(base=3, exponent=3)}')

# The order of keyword arguments becomes irrelevant
print(f'power(exponent=3, base=3)\t={power(exponent=3, base=3)}')

```

長方形面積: 17.6 平方單位

calculate_rectangle_area() missing 1 required positional argument: 'width'

power(3) = 9

power(3, 3) = 27

power(base=3, exponent=3) = 27

power(exponent=3, base=3) = 27

參數說明：

1. 僅限位置參數 (Positional-only parameters)

- 在參數清單中使用 / 標記
- 必須在函數呼叫時提供
- 不能使用關鍵字參數的形式傳入

2. 可選的位置/關鍵字參數 (Positional or keyword parameters)

- 位於 / 和 * 之間
- 可以有預設值
- 可以使用位置或關鍵字參數形式傳入

3. 僅限關鍵字參數 (Keyword-only parameters)

- 在參數清單中使用 * 標記
- 必須使用關鍵字參數形式傳入
- 提高程式碼可讀性，強制呼叫者明確指定參數名稱

這種參數設計方式在 Python 3.8+ 中特別有用，可以建立更清晰、更安全的 API。

```
In [57]: def process_data(name, age,/, sex='男', *, city):
          """處理使用者資料的函數

          這個函數展示三種參數類型：
          - 僅限位置參數 (name)
          - 可選的位置/關鍵字參數 (age)
          - 僅限關鍵字參數 (city)

          Args:
            name (str): 使用者姓名 (僅限位置參數)
            age (int, optional): 使用者年齡，預設為25 (位置參數)
            sex (str, optional): 使用者性別，預設為男 (可選的位置/關鍵字參數)
            city (str): 使用者所在城市 (僅限關鍵字參數)

          Returns:
            dict: 包含處理後的使用者資料
          """
          return {
              "name": name,
              "age": age,
              "sex": sex,
              "city": city,
              "is_adult": age >= 18
          }

# 正確的呼叫方式
# 1. 只提供必要參數
try:
    # 會出錯，因為 age 是必須的位置參數
    print(f'process_data("王小明", city="台北") = \n\t錯誤：缺少位置參數')
    result = process_data("王小明", city="台北")
except Exception as e:
    print(e)
```

```
# 2. 使用位置參數
# 預設的 'sex'
result1 = process_data("王小明", 30, city="台北")
print(f'process_data("王小明", 30, city="台北") = \n\t{result1}')

# 3. 使用關鍵字參數 (age)
try:
    # 會出錯，因為 'age' 是必須的位置參數
    print(f'process_data("王小明", age=30, city="台北") = \n\t錯誤: age')
    result2 = process_data("王小明", age=30, city="台北")
except Exception as e:
    print(e)

# 4. 錯誤: 缺少關鍵字參數 'city'
try:
    print(f'process_data("王小明", 30, "台北") = \n\t錯誤: 缺少關鍵字參數')
    result3 = process_data("王小明", 30, "台北")
except Exception as e:
    print(e)
```

```
process_data("王小明", city="台北") =
    錯誤: 缺少位置參數
process_data() missing 1 required positional argument: 'age'
process_data("王小明", 30, city="台北") =
    {'name': '王小明', 'age': 30, 'sex': '男', 'city': '台北', 'is
    _adult': True}
process_data("王小明", age=30, city="台北") =
    錯誤: age 非位置參數
process_data() got some positional-only arguments passed as keyword
arguments: 'age'
process_data("王小明", 30, "台北") =
    錯誤: 缺少關鍵字參數
process_data() missing 1 required keyword-only argument: 'city'
```

9. 模組 (Modules)

用下面兩個範例給你說明模組的使用：

1. os 模組
2. psutil 模組 (參見上面8.2節的自訂函式範例)
3. matplotlib, numpy 模組

語法 (Syntax):

```
import <模組名稱>
```

9.1 OS 模組

```
import os
```

In [58]: `import os`

```
In [59]: # get current working directory  
import os  
os.getcwd()
```

```
Out[59]: '/Users/michaelkao/My_Projects/Python'
```

```
In [60]: # List files in current directory  
import os  
os.listdir()
```

```
Out[60]: ['graphUSA50.py',  
          'my_module.py',  
          'hello.py',  
          '2. Python Tips and Tricks.pdf',  
          'sortDict.py',  
          'genExp.py',  
          '1. Python Tutorial TW.png',  
          'biTree.py',  
          'multiprocessing1.py',  
          'process_images_1P.py',  
          'csvEx2.py',  
          'xmlEx.py',  
          'arithmetic.py',  
          'tkinterEx2.py',  
          '1. Python Tutorial TW.ipynb',  
          'randomWalk.py',  
          'New Folder',  
          'video',  
          'gameSquareDays.py',  
          'unit_test',  
          'biTree_v1.py',  
          '2. Python Tips and Tricks.png',  
          'tupleEx1.py',  
          '1. Python Tutorial TW.pdf',  
          '2. Python Tips and Tricks TW.ipynb',  
          'process_images_CT.py',  
          '.DS_Store',  
          'primeLib.py',  
          'osModule.py',  
          'smsViaHTTP.py',  
          'bstEx2.py',  
          'setEx.py',  
          'csvEx.py',  
          'dictEx.py',  
          'logger',  
          'gameGuess4.py',  
          'guiEx.py',  
          'bsTreeEx.py',  
          'requirements.txt',  
          'heapBinary.py',  
          'uv.lock',  
          'perfComprehensions.py',  
          'generatorEx.py',  
          'employeeEx.py',  
          'perfHeaps.py',  
          'loopEx.py',
```

```
'process_images_CP.py',
'2. Python Tips and Tricks.html',
'smsViaSMTP.py',
'dunderEx.py',
'test_copy.txt',
'fibo.py',
'bsTree.py',
'loggingEx.py',
'eafp.py',
'jsonEx2.py',
'modEx3.py',
'iterEx2.py',
'xmlEx4.py',
'funcEx.py',
'sysPython.py',
'gameRollDice.py',
'bst.py',
'randomEx.py',
'filesEx6.py',
'graphEx5.py',
'piEstimate.py',
'gcd.py',
'slicingEx.py',
'gRatio.py',
'stairs.py',
'pyproject.toml',
'1. Python Tutorial TW.html',
'filesEx2.py',
'quake.py',
'perf_LvsT.py',
'gameSnake3.py',
'factorialEx.py',
'emailSendHTML.py',
'output',
'sortDict2.py',
'biTreeEx_v2.py',
'emailOnQuake.py',
'multiprocess10.py',
'2. Python Tips and Tricks.ipynb',
'filesEx3.py',
'2. Python Tips and Tricks TW.html',
'gameSnake2.py',
'pingSvr.py',
'perfDB.py',
'modEx2.py',
'iterEx3.py',
'__pycache__',
'sortObj.py',
'xmlEx5.py',
'piLib.py',
'greetings2.py',
'graphEx4.py',
'iterEx.py',
'bstEx.py',
'emailSendAttach.py',
'perfListvsGenerator.py',
```

```
'iterEx4.py',
'graphEx.py',
'1. Python Tutorial.png',
'listEx.py',
'README.md',
'biTreeEx_v1.py',
'xmlEx2.py',
'graphEx3.py',
'gameSRP.py',
'iterEx4_noflag.py',
'greetings.py',
'gameGuessNbr.py',
'SQLiteEx.py',
'jsonEx.py',
'scheduleEx.py',
'4. Python Advanced Topics2.ipynb',
'cacheEx2.py',
'people.py',
'brAsyncEx.py',
'1. Python Tutorial.pdf',
'3. Python Advanced Topics1.ipynb',
'.gitignore',
'heapBinomial.py',
'gameSnake5.py',
'condFlowEx.py',
'qrCodeGen.py',
'jsonAPI.py',
'.env',
'magicSquare.py',
'my_moduleEx.py',
'cacheEx.py',
'filesEx5.py',
'perfListvsTuple.py',
'graphEx6.py',
'.venv',
'gameSnake4.py',
'iterEx5.py',
'emailPublisher.py',
'modEx4.py',
'sample',
'.python-version',
'qt5sql.py',
'xmlEx3.py',
'brSyncEx.py',
'Python.code-workspace',
'graphEx2.py',
'xmlInspect.py',
'filesEx1.py',
'ProductionPythonCode',
'dates.py',
'go2Mars.py',
'img_processed',
'meminfo.py',
'emailSender.py',
'sortTuple.py',
'qt5Ex1.py',
```

```
'httpPing.py',
'gamePoker.py',
'environment.yaml',
'legbEx.py',
'tupleEx3.py',
'comprehensionEx.py',
'fastapiEx.py',
'.git',
'main.py',
'legbEx3.py',
'flagsEx.py',
'numberConversion.py',
'reEx.py',
'glogTime.py',
'biTreeEx.py',
'smsOnQuake.py',
'video-orig',
'comprehensionEx2.py',
'reEmails.py',
'data',
'advFormatting.py',
'csv_processor.log',
'template.env.txt',
'multiprocessing2.py',
'legbEx2.py',
'lambdaEx.py',
'multiprocessingEx.py',
'sieveEra.py',
'photo',
'1. Python Tutorial.html',
'2. Python Tips and Tricks TW.png',
'tkinterEx1.py',
'sortList.py',
'renameEx.py',
'git',
'2. Python Tips and Tricks TW.pdf',
'gameSnake.py',
'1. Python Tutorial.ipynb',
'mcp_server.py',
'mem_profile.py',
'heapFibo.py',
'daxia_rice_noodle_sign.py',
'biTree_v2.py',
'cacheLib.py',
'tupleEx2.py',
'exceptionEx.py']
```

```
In [61]: # 建立目錄 (參數：名稱, 模式, 是否允許已存在)
import os
os.makedirs("New Folder", exist_ok=True)
os.listdir("./New Folder")
```

```
Out[61]: []
```

9.3 行星拉格朗日點 (Lagrange Points)

拉格朗日點是兩個大天體（例如太陽與地球、地球與月球）在互相引力加上離心力的作用下，形成的一種特殊平衡位置。

把一個小物體（太空船、小行星、太空塵埃等）放在這些點上，它就能相對於這兩個大天體保持相對靜止，跟著它們一起繞圈，幾乎不需要額外推進力。

在太陽－行星系統中，總共有五個拉格朗日點，標記為 L1、L2、L3、L4、L5。

下面範例程式碼繪製日地系統拉格朗日點，這個視覺化清楚地展示了日地系統中的五個拉格朗日點位置，有助於理解這些特殊點在太空中的相對位置。

1. 基本設定：

- 設定太陽在原點 (0,0)，地球在 (1,0) AU 處
- 使用 AU (天文單位) 作為距離單位，1 AU = 地球到太陽的平均距離

2. 拉格朗日點計算：

- L1：位於地球朝向太陽的一側，約 0.99 AU
- L2：位於地球背向太陽的一側，約 1.01 AU
- L3：位於太陽另一側，約 -1 AU
- L4 和 L5：與太陽和地球形成等邊三角形

3. 視覺化：

- 藍色圓點：太陽
- 黃色圓點：地球
- 紅色 X：拉格朗日點
- 虛線圓：地球軌道

4. 圖表元素：

- 網格線：幫助讀取座標
- 圖例：說明各元素代表意義
- 座標軸標籤：標示距離單位
- 距離標記：X 軸上的 AU 標記

```
In [62]: import matplotlib.pyplot as plt
import numpy as np

# 設定中文字體
plt.rcParams['font.sans-serif'] = ['Arial Unicode MS', 'Microsoft J]
plt.rcParams['axes.unicode_minus'] = False # 正確顯示負號
#
# 太陽系座標設定
# 假設太陽在原點 (0,0)，地球在 (1,0)，距離單位為 AU (天文單位)
sun = (0, 0) # 太陽座標
earth = (1, 0) # 地球座標

# 拉格朗日點近似位置計算
```

```

# L1 ≈ 0.99, L2 ≈ 1.01, L3 ≈ -1 (相對於太陽的距離，單位AU)
# L4 和 L5 形成等邊三角形，與太陽和地球夾角60度
mu = 3e-6 # 地球與太陽的質量比，約為 3×10-6

# 計算拉格朗日點位置
L1_approx = 1 - (mu/3)**(1/3) # L1 點位置，約 0.990 AU
L2_approx = 1 + (mu/3)**(1/3) # L2 點位置，約 1.010 AU
L3_approx = -1 # L3 點位置，在太陽另一側

# 計算 L4 和 L5 點座標 (等邊三角形頂點)
theta = np.deg2rad(60) # 60度轉換為弧度
L4_x = 0.5 # L4 點的 x 座標
L4_y = np.sin(theta) # L4 點的 y 座標
L5_x = 0.5 # L5 點的 x 座標
L5_y = -L4_y # L5 點與 L4 點對稱於 x 軸

# 創建圖形
fig, ax = plt.subplots(figsize=(9, 7))

# 繪製太陽和地球
ax.plot(sun[0], sun[1], 'bo', markersize=20, label='太陽 (Sun)')
ax.plot(earth[0], earth[1], 'yo', markersize=12, label='地球 (Earth)')

# 繪製地球軌道 (圓形近似)
theta_orbit = np.linspace(0, 2*np.pi, 200)
ax.plot(np.cos(theta_orbit), np.sin(theta_orbit), 'b--', lw=0.8, alpha=0.5,
        label='地球軌道 (Earth\'s Orbit)')

# 定義五個拉格朗日點的座標和標籤
lagrange_points = [
    (L1_approx, 0, 'L1'), # 第一拉格朗日點
    (L2_approx, 0, 'L2'), # 第二拉格朗日點
    (L3_approx, 0, 'L3'), # 第三拉格朗日點
    (L4_x, L4_y, 'L4'), # 第四拉格朗日點
    (L5_x, L5_y, 'L5') # 第五拉格朗日點
]

# 繪製拉格朗日點
for x, y, label in lagrange_points:
    ax.plot(x, y, 'x', markersize=8, color='red')
    # 調整標籤位置
    if label == 'L1':
        ax.text(x-0.1, y+0.08, label, fontsize=12, color='blue',
                bbox=dict(facecolor='white', alpha=0.7, edgecolor='black'))
    else:
        ax.text(x+0.02, y+0.08, label, fontsize=12, color='red',
                bbox=dict(facecolor='white', alpha=0.7, edgecolor='black'))

# 添加圖表標題和標籤
ax.set_title('日地系統拉格朗日點 (Sun-Earth Lagrange Points)', fontsize=14)
ax.set_xlabel('X 座標 (AU)', fontsize=12)
ax.set_ylabel('Y 座標 (AU)', fontsize=12)

# 設置座標軸範圍和比例
ax.set_xlim(-1.5, 1.5)
ax.set_ylim(-1.2, 1.2)

```

```

ax.set_aspect('equal') # 確保 x 和 y 軸比例相同

# 添加網格和圖例
ax.grid(True, alpha=0.3)
ax.legend(loc='upper right')

# 添加參考線
ax.axhline(0, color='k', linestyle='--', alpha=0.1)
ax.axvline(0, color='k', linestyle='--', alpha=0.1)

# 添加距離標記
for x in np.arange(-1.5, 2.0, 0.5):
    ax.axvline(x, color='gray', linestyle=':', alpha=0.2)
    if x != 0: # 跳過原點
        ax.text(x, -1.3, f'{x:.1f} AU', ha='center', va='top', font:

# 顯示圖形
plt.tight_layout()
plt.show()

```

日地系統拉格朗日點 (Sun-Earth Lagrange Points)

